Learning versus Planning for Orbit Wars: ctx2, a Hybrid Learned Re-Ranker, and NeRL, a Model-Predictive Planner

Vũ Nguyên Đan, Phạm Tiến Dũng, Lê Hồng Anh
Group 12 — Team `IAI-RL-DirtyDozen`
Student IDs: 23020351, 23020345, 23020327
Submitting representative: Vũ Nguyên Đan (23020351)
Course: Reinforcement Learning and Planning
University of Engineering and Technology – Vietnam National University, Hanoi (UET–VNU)
Hanoi, Vietnam

***Abstract*—This report describes two different agent designs for the real-time strategy game *Orbit Wars*, both built by our team. The first, *ctx2*, is a hybrid agent: a rule-based engine handles all physics and game mechanics (candidate generation, fleet sizing, orbital lead-aiming, sun avoidance, combat resolution, defensive reinforcement), while the most important strategic decision—*which planet to attack and from where*—is handled by a small neural network that acts as a *re-ranker*. The network reads 230 features per candidate (46 base features plus DeepSet-style set statistics) and uses an MLP of size $230 \to 128 \to 128 \to 1$. It is trained by supervised regression on about 510 000 labelled candidates and exported to plain Python so it runs within the one-second-per-turn limit. On a held-out split it picks the same top candidate as the label about 81–85% of the time; its public score is about 1001 (current entry 1000.8; it was near 1067 earlier).**

The second agent, NeRL (short for *Not even RL*), is a model-predictive planner. It has *no* learned parameters. It builds a forward model of the game using tensors, predicts each planet's future owner and garrison (the ships stationed on it) over a planning horizon, and chooses launches by how much they improve that predicted state (a competitive ship-flow objective). It then selects several attack waves greedily, sends the minimum fleet needed to capture (the capture floor), combines fleets from several planets when one is not enough (focus-fire), and regroups ships for defence. In direct play it beats both *ctx2* and our strongest heuristic decisively. As our active leaderboard submission it scores 1141.8, above the re-ranker. We explain why a planner with an exact model avoids the imitation ceiling and distribution shift that limit the supervised re-ranker, and what this means for combining learning with planning.

***Index Terms*—Orbit Wars, model-predictive planning, receding horizon, supervised learning, re-ranker, DeepSet, game AI, self-play**

I. Team, contributions, and repository

A. Group and members

This report is submitted by **Group 12** on the class list, under the team name `IAI-RL-DirtyDozen`. The members and student IDs are:

- Vũ Nguyên Đan (23020351) — *submitting representative*
- Phạm Tiến Dũng (23020345)
- Lê Hồng Anh (23020327)

B. Work division and contributions

In the first week (25/05–31/05) each member worked on a separate branch and explored a different approach. From 01/06 to 09/06 we focused on the strongest line: we took the work of the member with the highest leaderboard score as the base and built on it, adjusting as scores changed. Table I lists the roles and contributions.

Table I
Member roles and contributions

Member	Role and contribution	Share
Vũ Nguyên Đan	Team lead; *ctx2* agent; report writing	42.5%
Phạm Tiến Dũng	*NeRL* agent; report writing	42.5%
Lê Hồng Anh	Active in the first week; not active in the final week	15%

C. Repository

All code—both agents, the training pipeline, the evaluation harness, and the source of this report—is at:

`https://github.com/dxpawn/OrbitWars`

D. Leaderboard standing (summary)

At the time of writing, `IAI-RL-DirtyDozen` has a public score of **1141.8**, a global rank of **636 / 4106** teams, and is **12th of the 19** `IAI-RL-*` class teams. The full class breakdown is in Sec. VI-C, Table V.

II. Introduction

Orbit Wars is a two-dimensional real-time strategy problem: planets orbit a central sun, each one produces resources and holds ships, and players send fleets to capture planets. Each turn an agent decides how many ships to send, from which planet, to which target, balancing expansion, defence, and attack. The action space is large and the orbital motion is

complex, so a full global search within the per-turn time limit is not feasible.

We approached the problem in two ways, and this report presents both as one comparison.

Approach A — learning where it matters (*ctx2*). We separate *mechanics* from *judgement*. Everything that can be written as a rule—fleet sizing, aiming, sun avoidance, combat, reinforcement—is handled by a rule engine. Only the high-value decision, "which of the feasible targets is worth attacking," is learned from collected data. This follows a simple principle: do not use machine learning where a reliable rule already works; spend the model only on the most valuable decision.

Approach B — planning with an exact model. Instead of *learning* a target preference, the planner *computes* one. Because the game's rules are known, we simulate the near future exactly and choose the launches that move the predicted result most in our favour. This is model-based planning in the receding-horizon / model-predictive-control sense [4], [5]: the model is *given*, not learned.

Contributions.

1) A hybrid engine-plus-re-ranker agent (*ctx2*) with a DeepSet-style permutation-invariant set encoding, trained by offline supervised learning and exported to dependency-free Python (Sec. IV).
2) A tensor-based model-predictive planner that predicts garrison ownership forward and scores launches by their marginal effect on a competitive ship-flow objective, with capture-floor ship economy, focus-fire, and re-grouping (Sec. V).
3) An honest comparison of the two approaches, covering the imitation ceiling, distribution shift, and the limited value of local self-play as a ranking signal (Sec. VII).

III. Problem and game mechanics

The board is 100×100 with a sun of radius $R_{\text{sun}} = 10$ at the centre. Planets can be static or move on orbits; the maximum fleet speed is 6 units per turn and a match lasts up to 500 turns. Each turn the agent receives an *observation* with every planet (owner, ship count, production, position), all fleets in flight, and incoming threats. Fleet speed depends on fleet size; a fleet aimed at a moving planet must lead its target; and a fleet is destroyed if its path crosses the sun. To capture a planet, a fleet must arrive with more ships than the defender has at the moment of arrival; after that, the planet's production goes to the new owner.

These rules are fully known and deterministic once the players' actions are fixed. This is what makes Approach B possible: a known transition function can be *simulated* instead of *learned*.

IV. Approach A: hybrid engine and learned re-ranker (ctx2)

A. Scope of the learned component

ctx2 does **not** learn a full policy. The network is used only at the *re-ranking* step: after the engine lists the feasible attack candidates for each owned source planet, the network gives each one a quality score and adjusts the priority order. Everything else—the fleet size needed to win and hold, leading a moving target, avoiding the sun's shadow, counter-snipe handling, reinforcing threatened planets—is done by the rule engine in `engine/orbit_base.py`.

B. Per-turn decision pipeline

The pipeline has six steps:

1) **Parse the observation.** Decode the state into planets (owner, ships, production, orbital position), fleets in flight, per-side totals, and incoming threats.
2) **Generate candidates.** For each owned planet, the engine lists targets it can reach, pre-sorted by a weighted-distance heuristic. Each candidate's position in the merged list is recorded as the index `idx`. An expansion factor `expand` $= 10$ keeps extra candidates before cutting to the top K.
3) **Build features.** Each (source, target) pair is turned into a 46-dimensional vector (Sec. IV-E).
4) **Score with the network.** `score_many(rows)` takes all candidates of one call, adds set statistics (mean, max, min, std per dimension), normalises, and runs the MLP.
5) **Re-rank.** The final priority is
$$\text{priority} = \text{idx} - \text{bonus} \cdot \sigma(\text{score}), \tag{1}$$
where σ is the sigmoid and bonus $= 1.45$ (2p) or 1.25 (4p). A high learned score moves a candidate up from its heuristic order; `bonus` caps how much the network can change the order. In 4p a small jitter (10^{-6}) breaks ties.
6) **Execute.** The engine sends fleets to the chosen targets: it computes the ships needed to win and hold, leads the moving target, avoids the sun, and runs a defensive reinforcement pass.

Network inference is about **0.1 s/turn**, well inside the one-second limit.

C. The rule engine

The engine in `orbit_base.py` contains most of the rule-based game logic:

- **Fleet sizing and navigation:** speed as a function of fleet size, predicting the target's position at arrival (lead aiming), sun-collision checks.
- **Initial target heuristic:** weighted distance, a preference for static planets, an expansion bonus, and handling of the case where several sides target the same planet.
- **Short-horizon forward projection (4p):** a 7-turn look-ahead that adjusts candidate ranking for value beyond one step.
- **Active defence:** snipe and counter-snipe detection, reinforcing weak planets, late recapture in 2p.
- **Multi-action control:** a per-turn action cap (7 in 2p, 5 in 4p) and a source-expansion search.

Separating the engine from the network helps debugging: the mechanics and candidate generation can be tested on their

own, and the re-ranker is enabled only when evaluating the learned part.

D. Network architecture

The re-ranker is a three-layer MLP with ReLU activations, trained by supervised regression to predict a target-quality score from the feature vector. After training, the weights are exported to plain Python—no PyTorch or TensorFlow at competition time.

a) 230-dimensional input and DeepSet encoding: Each candidate i in a decision has a base vector $\mathbf{x}_i \in \mathbb{R}^{46}$. Before the MLP, we add set statistics over all candidates S of that decision:

$$\boldsymbol{\mu} = \text{mean}_{j \in S}(\mathbf{x}_j), \qquad \mathbf{m} = \max_{j \in S}(\mathbf{x}_j), \tag{2}$$

$$\mathbf{n} = \min_{j \in S}(\mathbf{x}_j), \qquad \boldsymbol{\sigma} = \text{std}_{j \in S}(\mathbf{x}_j), \tag{3}$$

(each computed per dimension). The full input is $\mathbf{z}_i = [\mathbf{x}_i; \boldsymbol{\mu}; \mathbf{m}; \mathbf{n}; \boldsymbol{\sigma}] \in \mathbb{R}^{230}$.

This is a **DeepSet-style set-pooling** [1]: the network judges a candidate *relative* to the other options in the same decision, not on its own, while still scoring each candidate cheaply and one at a time. The model is **permutation-invariant** over candidate order and handles a varying number of candidates per turn.

b) Forward pass: After normalisation $\hat{\mathbf{z}} = (\mathbf{z} - \boldsymbol{\mu}_{\text{train}})/\boldsymbol{\sigma}_{\text{train}}$ (the `MEAN` and `STD` constants are stored in the weight file):

$$h_1 = \text{ReLU}(W_0\hat{\mathbf{z}}+b_0),\ h_2 = \text{ReLU}(W_1 h_1+b_1),\ s = W_2 h_2+b_2. \tag{4}$$

The architecture is $230 \xrightarrow{\text{ReLU}} 128 \xrightarrow{\text{ReLU}} 128 \xrightarrow{\text{linear}} 1$. The scalar s, after a sigmoid, is used in Eq. (1). We keep two separate weight sets (`student_weights_2p.py`, `student_weights_4p.py`) because the state distribution and the re-rank settings differ between 2p and 4p.

E. The 46 base features

The features are built in `engine/main.py` and combine global state (shared by all candidates in a turn) with local (source, target) information. Most values are clamped to $[0, 1]$ or $[-1, 1]$; ship counts pass through a saturating map $x/(x + 40)$ to reduce the effect of absolute scale. Table II lists all 46 dimensions in order.

A short history layer (`_FeatureHistory`) keeps a 5–10 turn buffer to compute *momentum*, *aggression_trend*, *enemy_rhythm*, *approach_rate*, and *convergence_threat*—trend features that let the network sense the game phase and opponent pressure instead of a single snapshot. The re-rank settings are in `engine/feature46_manifest.json`: `bonus_2p=1.45`, `bonus_4p=1.25`, `expand=10`.

F. Training method

The model is trained by **offline supervised learning**: collect (features → quality-score) pairs, then fit them. There is no environment reward and no exploration—only MSE regression onto fixed labels.

Quality labels. For each decision, an internal scoring pipeline ranks every candidate from its 46 features and set context. This score is the strategic priority assigned by the reference scorer, and it is the regression target for the smaller MLP.

Data collection.

- Run the agent over about **250 games** (2p and 4p) against a mixed pool (`heuristic_v6`, `adv_hellburner`, `adv_proto_v15`, `adv_lb958`).
- Each `score_many` call is tagged with a *group id*; all candidates of a call share a group, so the set context can be rebuilt at training time.
- Size: about 25 000 decision groups (2p), about 37 000 (4p); about **510 000** labelled candidates in total.

Training.

- Join the 46 features with the group's mean/max/min/std $\Rightarrow$ 230 dimensions; normalise by the global mean and std.
- Split 90/10 *by group* (not by row) to avoid leakage between train and validation.
- MSE loss between predicted and target score; Adam optimiser on GPU; about **1 minute** of training.
- Main metric: *held-out group top-1 agreement*—in each validation group, does the model's top candidate match the label?

Export. The weights, `MEAN`, and `STD` are written to `model/student_weights_{2p,4p}.py`; `score_many` rebuilds the set-pooling and forward pass exactly as in training.

V. Approach B: NeRL, a model-predictive planner

NeRL—short for *Not even RL*, because it does not learn at all—plays the same game as *ctx2* but takes the opposite approach: instead of *learning* which target to prefer, it *simulates* the result of every candidate launch and picks the one that improves the predicted outcome the most. The name reflects that NeRL belongs to the *planning* part of the course (model-based planning with a known model), not the learning part. It has **no neural network and no learned weights**; `torch` is used only as a fast array library so the whole simulation fits in the time budget.

A. What the agent is, in RL terms

Because the rules are known exactly, NeRL is **model-based planning with a given model** [5]—a receding-horizon, or model-predictive, controller [4]. It is *not* a reactive heuristic (it does not score the current state; it simulates the future), and it is *not* learned. Its only heuristic parts are practical approximations that keep planning fast: shortlisting candidates, greedy (not optimal) selection, a fixed horizon, and the assumption that opponents launch nothing new beyond fleets already in flight (no adversarial game-tree search).

B. Per-turn pipeline

The algorithm in `run_turn`:

1) **Parse** the observation into tensors (planets, fleets in flight).

Table II
THE 46 BASE FEATURES (ORDER AS IN _CANDIDATE_FEATURES)

#	Name	Short description
Global / macro state (indices 0–11)		
0	game_progress	Current turn / 500
1	players_alive	Number of active players / 4
2	my_planet_share	Fraction of planets we own
3	my_gdp_share	Fraction of total production we own
4	my_ship_share	Fraction of all ships (planet + transit) we own
5	momentum	Change in our ship share vs. 5 turns ago
6	my_fleet_ratio	Ships in transit / our total ships
7	leader_gap	Normalised force gap to the leading side
8	am_i_targeted	Fraction of enemy fleets aimed at our planets
9	fastest_grower_gap	Momentum gap between the fastest-growing rival and us
10	aggression_trend	Trend of being targeted (5 turns)
11	enemy_rhythm	Variation in enemy attack tempo (10 turns)
Source planet and diplomacy (12–19)		
12	src_ships	Source ship count (saturating)
13	src_production	Source production / 5
14	src_safety	Source safety against incoming threats
15	diplomacy	$+1$ our defence, 0 neutral, -1 attacking a rival
16	target_production	Target production / 5
17	is_dynamic_target	1 if comet or non-static planet
18	owner_power	Target owner's ship share of the board
19	is_weakest_enemy	1 if target belongs to the weakest rival
Capture and threat simulation (20–31)		
20	proj_owner	Predicted owner at arrival: us / neutral / enemy
21	proj_ships	Predicted defender ships at arrival (saturating)
22	threat_after	Enemy threat arriving *after* we capture
23	support_after	Our support arriving after we capture
24	pre_volatility	Force volatility before we arrive
25	threat_potential	Attack potential from other enemy planets
26	support_potential	Support potential from our other planets
27	eta	Time to arrival / 25
28	send_fraction	Ships sent / our total ships
29	need_ratio	Ships needed to win / source ships
30	margin	Safety margin (send $-$ need), normalised
31	can_win	1 if we send enough ships to capture
Strategy and terrain (32–45)		
32	garrison_strength	Remaining force after capture (saturating)
33	defense_sustainability	Ability to hold after capture
34	target_roi	Target production / (ships needed $+1$)
35	front_diff	Distance-to-nearest-enemy gap (source vs. target)
36	avg_enemy_distance	Mean distance to the enemy centroid
37	angular_spread	How much enemies surround us by angle
38	orbital_trend	ETA trend due to orbital motion
39	convergence_threat	Convergence threat (enemies closing distance)
40	approach_rate	Rate enemies approach our centroid
41	enemy_commitment	Enemy fraction of ships in transit
42	weakest_colony	Our weakest planet (after subtracting incoming threats)
43	local_superiority	Local support-vs-threat advantage
44	economic_impact	Target production / our production
45	enemy_exposed	How exposed the enemy side is

2) **Build or refresh the movement model.** `PlanetMovement` predicts every planet's position over the horizon H, tracks fleets in flight with a ledger, and supports forward prediction of garrisons.
3) **Distance cache.** Cross-time distances "planet s at t=0 to planet t at step k"—the geometrically correct distance for checking whether a fleet can reach a moving planet in time.
4) **Garrison prediction.** `garrison_status` returns, for steps $0 \dots H$, each planet's predicted (owner, ships) using an *exact* recurrence: production, fleet arrivals, and combat resolved step by step.
5) **Plan waves** (Sec. V-C).
6) **Emit** the chosen launches as a sparse action payload.

C. Wave planning and scoring

a) Shortlists: Sources are owned planets with at least `min_ships_to_launch` ships, ranked by ship count. Targets combine an *offensive* shortlist (enemy or neutral planets by distance) and a *defensive* shortlist of *flip targets*—our own planets predicted to be lost within the horizon, ranked by urgency $=$ prod $\cdot$ (turns left) $+$ ships.

b) Fleet sizing and reachability: For each (source, target), the launch size is `safe_drain`: the most ships the source can send while still holding itself over the horizon. An analytic reachability test (which accounts for orbital drift, the sun blocking the line of sight, and surface offsets) and an `intercept_angle` solver (fixed-point iteration) give the aim angle and ETA to the moving target.

c) Capture floor: `capture_floor` computes the *minimum* ships needed to take a target at its arrival turn:

$$f_k = \left\lceil \text{defenders}_k + \text{overhead} + \text{reinforcements}_k \right\rceil, \quad (5)$$

where defenders_k is the predicted garrison at arrival step k. A candidate is kept only if its size clears f_k. This gives efficient ship use: the agent never sends more than needed for a capture.

d) Focus-fire: When no single source clears a target's floor, the planner *combines* several sources whose fleets arrive on the *same* turn and whose total ships exceed f_k, forming a coordinated multi-source strike. This is the main feature the strongest configuration adds over a single-source planner.

e) Marginal competitive scoring: Each candidate launch set c is scored by its effect on the predicted ship flow. Let $\Delta\eta_j(c)$ be the change in player j's predicted *net* ships (produced minus lost in combat over the horizon) caused by c. The objective is

$$s(c) = \Delta\eta_{\text{me}}(c) \; - \sum_{j \neq \text{me}} \Delta\eta_j(c). \quad (6)$$

This is a one-step (one-ply) look-ahead over the exact simulator: it rewards launches that raise our net ships and lower the opponents' net ships.

f) Greedy selection and regroup: A greedy loop picks the highest-scoring non-conflicting waves, up to `max_waves_per_turn`, within each source's budget and above an ROI threshold (fire only if $s(c) >$ `roi_threshold`). Finally, a *regroup* pass moves spare ships from safe planets toward threatened friendly planets, following an enemy-pressure gradient that includes enemy fleets in flight.

D. Configurations

A fixed `ProducerLiteConfig` holds the settings. The 2p preset uses horizon $= 18$, up to 6 waves per turn, ROI $=$ 1.5, and focus-fire on (up to 4 strike sources). The 4p preset shortens the horizon to 13 and reduces the number of sources, defensive targets, and strike sources, because a four-player game has more branching and a shorter useful look-ahead.

VI. Experimental results

A. ctx2: held-out label agreement

Table III
CTX2 TRAINING METRICS ON THE HELD-OUT (BY-GROUP) SPLIT

Metric	2p	4p
Group top-1 agreement	81.3%	84.8%
R^2 (MSE on scores)	0.917	0.933
Overall: top-1 ≈85%, top-3 ≈95%, $R^2 \approx 0.93$		

Top-1 agreement of about 85% means that in about 85% of re-rank decisions the model's top candidate matches the label; top-3 of about 95% means the highest-labelled target is usually in the model's top three, so the errors are mostly in secondary ordering rather than missing the key target. The public score is about **1000.8** (it was near 1067 earlier; TrueSkill scores drift by about ± 30). Inference is about 0.1 s/turn, and the submission needs only standard Python at competition time.

B. NeRL beats our other agents head-to-head

In direct play (our local harness, built on `kaggle_environments`) NeRL is clearly stronger than both our heuristic and *ctx2*:

Table IV
NERL VS. OUR OTHER AGENTS (2P, REPRESENTATIVE SEEDS)

Match (NeRL = P0)	Winner	Score	Turns to win
NeRL vs. `heuristic_v6`	NeRL	1211–0	85 / 500
NeRL vs. *ctx2*	NeRL	1478–0	94 / 500

In both 2p matches NeRL eliminates the opponent before turn 100 of 500. As our active leaderboard submission it scores **1141.8**—above *ctx2*'s 1000.8 and consistent with the head-to-head result, but only in the middle of the full 4106-team field (Sec. VI-C). The two numbers measure different things: the head-to-head result is a direct duel against our own agents, while the leaderboard score is a rating against the whole field. Per-turn compute (`torch` on CPU) stays well under the one-second limit even though it runs a full forward prediction every turn.

C. Leaderboard standing

Table V shows our team among the `IAI-RL-*` class teams on the public Orbit Wars leaderboard (snapshot of 09/06; TrueSkill scores drift by about ± 30 and keep changing as submissions keep playing). Our active submission is **NeRL** at 1141.8; our *ctx2* submission scored 1000.8. Among 4106 teams overall we rank 636th; among the 19 class teams we rank 12th. We state this plainly: NeRL beats our *own* earlier agents head-to-head, but across all teams it ranks in the middle, and several class teams using similar strong public agents rank above us. The value of this project is therefore less the score itself and more the comparison of *why* planning beats our learned re-ranker (Sec. VII).

Table V
`IAI-RL-*` CLASS TEAMS ON THE PUBLIC LEADERBOARD (09/06 SNAPSHOT, 4106 TEAMS TOTAL)

Class #	Team	Score	Global rank
1	IAI-RL-04	1294.9	79
2	IAI-RL-IX	1243.8	143
3	IAI-RL-Tlm71	1235.6	158
4	IAI-RL-MT	1224.9	207
5	IAI-RL-codex_v1	1200.7	314
6	IAI-RL-NapNap	1197.5	341
7	IAI-RL-Beginner	1197.2	342
8	IAI-RL-codex	1187.9	399
9	IAI-RL-03	1186.4	408
10	IAI-RL-17	1183.3	430
11	IAI-RL-II	1155.3	591
12	**IAI-RL-DirtyDozen (ours)**	**1141.8**	**636**
13	IAI-RL-Checkm8	1141.4	638
14	IAI-RL-003	1120.4	697
15	IAI-RL-Cherry_11	1119.4	700
16	IAI-RL-HaiPhong	969.7	926
17	IAI-RL-06	923.0	1022
18	IAI-RL-10	803.6	1421
19	IAI-RL-Trauma07	709.0	1906

VII. Discussion: learning vs. planning

A. The ceiling of offline supervised learning

ctx2's goal is to *match a label*, not to *win more games*. Our experiments showed that higher label agreement does not always mean stronger play—a version that fit the labels more closely actually scored *lower*—so **label agreement is not the same as competitive strength**. A supervised re-ranker can at best reach the quality of the scorer that made its labels; it cannot exceed it [3]. This is the ceiling the planner avoids: by optimising an *outcome* (predicted ship flow) instead of imitating a *preference*, the planner is not limited by any teacher.

B. Why the planner wins

The planner has three structural advantages over the re-ranker:

1) **It optimises outcomes, not labels.** Eq. (6) is based on the actual game dynamics; the re-ranker is based on a fixed scoring heuristic.
2) **Efficient ship use.** The capture floor sends the smallest winning fleet, which frees ships for more targets—an efficiency the feature-based re-ranker only approximates.
3) **Coordination.** Focus-fire runs multi-planet strikes that a re-ranker scoring each launch on its own cannot represent.

C. Distribution shift and compounding errors

At training time, *ctx2*'s data lies on the state distribution produced by the collection pipeline; at competition time, the agent follows its *own* distribution, and the mismatch causes **compounding errors**. The standard fix is **DAgger** [2]: relabel the states the agent actually visits and retrain. The planner does not have this problem—it re-plans from the true state every turn, so there is no train/test distribution gap.

D. Reliability of evaluation

Local self-play and win rate against a fixed pool do *not* match the public leaderboard closely, because of distribution shift to the real field of competitors and TrueSkill drift. We therefore trust only **paired held-out metrics** (agreement, R^2) and **direct leaderboard results**, and we treat any cross-day comparison of absolute scores with caution.

E. Limits and future work

The planner's biggest simplification is that it does not model the opponent: it assumes rivals launch nothing new beyond fleets already in flight. Useful extensions toward opponent modelling are (i) a shallow adversarial roll-out (simulate the strongest rival's likely best reply), (ii) replacing greedy wave selection with a small beam search, and (iii) using *ctx2*'s learned score to break ties inside the planner's shortlist—a concrete way to combine the two approaches, with learning suggesting candidates and planning making the final choice.

VIII. Implementation structure

Table VI shows the packaged model bundle; the full development repository—both agents, the training pipeline, and the evaluation harness—is at the GitHub link in Sec. I.

Table VI
PACKAGED SUBMISSION LAYOUT (CTX2 MODEL BUNDLE); NERL SHIPS AS ONE STANDALONE FILE

Path	Role
Approach A — ctx2 (learned re-ranker)	
`agent/submission_distilled_ctx2.py`	Single-file competition submissi
`model/student_weights_{2p,4p}.py`	MLP weights + `score_many`
`engine/orbit_base.py`	Rule engine (gameplay)
`engine/main.py`	Candidate generation, 46 feature
`engine/feature46_manifest.json`	Feature schema + re-rank settin
`training/distill_collect_grouped.py`	Grouped data collection
`training/distill_train_ctx.py`	Train + export
`training/validate_student.py`	Held-out agreement check
`training/selfplay.py`	A/B self-play
Approach B — NeRL (model-predictive planner)	
`agents/NeRL.py`	NeRL planner, single file

Reproduction (Approach A): collect → train → validate → build submission; the ≈85 MB `.npz` data can be rebuilt by the collection script. Approach B has no training step—the agent is self-contained and deterministic for a given seed.

IX. Conclusion

We presented two agents for Orbit Wars and an honest comparison of the two approaches. *ctx2* is a hybrid design—a rule engine for mechanics and a small DeepSet re-ranker (230-128-128-1) for target selection—trained on about 510 000 candidates to about 81–85% top-1 agreement and $R^2 \approx 0.92$–0.93, with a score of about 1000.8. NeRL, our model-predictive planner, has no learned parameters yet scores 1141.8 on the public leaderboard—above *ctx2*—and beats both *ctx2* and our heuristic head-to-head, because it optimises the simulated *outcome* instead of imitating a fixed preference. The broad lesson is the usual learning-versus-planning trade-off: when an exact model is available, planning with it can beat learning to imitate, and the most promising direction is to let learning suggest candidates and let planning make the final choice.

Acknowledgement

We thank the course instructors and the Orbit Wars organisers for the competition environment, benchmarks, and guidance, and the competitor community for real feedback on agent performance.

References

[1] M. Zaheer *et al.*, "Deep sets," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," *AISTATS*, pp. 627–635, 2011.

[3] A. Hussein *et al.*, "Imitation learning: A survey of learning methods," *ACM Computing Surveys*, vol. 50, no. 2, pp. 1–35, 2017.

[4] C. E. Garcia, D. M. Prett, and M. Morari, "Model predictive control: Theory and practice—a survey," *Automatica*, vol. 25, no. 3, pp. 335–348, 1989.

[5] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.